

AI view: 视频面试-初试-景语亮-UE客户端开发工程师-RD02

on Jul 23, 2026

Title: 视频面试-初试-景语亮-UE客户端开发工程师-RD02

Time: Jul 23, 2026 (Thu) 14:51 - 15:45 (GMT+08)

Participants:  jingyuliang @李智超

AI-generated content might be inaccurate. Please use with caution.

- 大厅启动性能优化: 针对《圆梦之星》大厅启动耗时过长问题, 通过 trace 分析耗时点, 将泳池、摩天轮等非必要子关卡改为分批加载, 启动时仅加载广场等关键地图, 其余内容延迟到进入大厅后加载
- 常规性能问题处理: 修复非 runtime 阶段提前 require 大体积 UObject 导致的内存泄漏问题, 优化高复杂度 for 循环等代码热点
- 虚幻引擎垃圾回收认知: 了解 UE 采用标记清扫机制, 从关键根节点出发遍历引用链, 释放无引用依赖的资产



业务开发技术栈与实践

代码语言占比与分工: 业务开发代码 80% 使用 Lua 编写, 涉及每帧执行的性能敏感逻辑 (如碰撞检测、道具打击判断) 则使用 C++ 实现, 非高频执行的广场活动、NPC 交互等逻辑均在 Lua 层完成



Lua 使用常见问题: 初期因不熟悉语法糖, 常出现点号与冒号使用错误导致未正确传递 self 参数; 闭包使用中存在 upvalue 被内部函数捕获后无法释放的问题, 但实际遇到的相关场景较少

UE 碰撞检测常用方式: 主要包含三种, 分别为可精确控制检测帧的主动 trace 检测、适合区域进入类低精度场景的 Overlap 碰撞盒检测、用于扫过区域碰撞判断的 sweep 检测, 未深入研究底层几何体相交的具体实现算法

虚幻引擎主流碰撞检测方案对比		
检测方式名称	核心特性	适用场景
主动 Trace 碰撞检测	主动触发、可自定义碰撞通道、检测时机可控、精度高	需精确控制碰撞的场景 (如道具开发)
Overlap 碰撞盒检测	被动触发、低精度	区域触发类场景 (如进入区域显示按钮)
Sweep 扫描碰撞检测	检测扫描路径内的碰撞	物体移动路径碰撞判定场景

引擎学习与对比情况: 本科期间系统学习过 Unity 并参与官方培训获奖, 工作后转用虚幻引擎未接受系统培训; UE 相比 Unity 原生功能更完善, 自带完整 Gameplay 框架、行为树与动画蓝图, 而 Unity 移动功能需依赖第三方插件, 状态机逻辑与 UE 存在差异

Unity 与 UE 引擎核心使用差异对比				
引擎类型	原生功能支持程度	Gameplay 框架实现	行为/逻辑处理方式	动画系统特性
Unity	弱, 需第三方组件/插件	需自行引入第三方移动组件	状态机	无动画蓝图
UE	强, 原生功能丰富	自带完整原生 Gameplay 框架	行为树	有动画蓝图

技术学习路径: 无系统性的虚幻引擎技术书籍学习习惯, 遇到动画蓝图等陌生需求时, 通过查阅官方文档、B 站及 YouTube 视频进行针对性学习

AI 辅助游戏开发实践与认知

- AI 开发核心优势: 可快速分析源码、讲解底层实现逻辑, 辅助开发者快速上手陌生模块, 同时在底层结构与规范完善的前提下, 未来可支持自然语言开发业务逻辑
- AI 开发现存问题: 会导致开发者手写代码熟练度下降, 且初始生成的代码普遍存在问题, 无法完全自由发挥
- AI 开发落地流程: 先由人工完成需求分析、与团队对齐确认方案, 再使用 Cursor 的 Plan 模式生成实现方案, 人工优化方案至合格后再生成代码, 最终通过本地验证、添加断言与日志等安全保障代码完成校验

《圆梦之星》角色多端同步实现逻辑

主控端移动与纠错机制: 主控端玩家输入直接本地执行保障跟手感, 每次移动生成唯一 ID 上传至 DS 端, DS 端根据冲量、方向等参数计算标准位置并与客户端上传位置校验, 出现偏差时下发纠错包修正角色位置



延迟补偿处理方案: 针对客户端到 DS 再回传的延迟问题, 采用重播逻辑, 先将角色移动到纠错包对应的正确位置, 再按顺序重新执行后续未处理的移动操作, 避免玩家操作丢失

招聘考察要点与发展建议

- 核心考察维度: 优先考察岗位能力匹配度, 针对刚毕业的候选人重点关注自学能力, 游戏热爱度为基础的默认加分项
- 后续学习建议: 针对意向的技术方向, 需私下多研究、学习网上成熟的开源项目以补充相关能力

候选人求职相关情况

- 候选人未进行海报, 仅投递自身认可、有发展潜力的意向公司
- 本次面试是候选人投递后较早安排的面试场次